



Module 4 Day 1: System Management Basics

Class Overview

Today, you will work through a series of command line exercises in which you will explore a Linux file system, collect evidence of a security breach, audit and reconfigure a Linux machine's user and group permissions, and take steps to harden the system.

Class Objectives

By the end of class, you will be able to:

- Manage processes with the `top`, `ps`, and `kill` commands.
- Install packages by using `apt install`.
- Use newly installed programs such as `terminator` as a network professional.

Lab Environment

You will use the Ubuntu VM inside your web lab for today's activities.

- Student credentials:
 - Username: `student`
 - Password: `cybersecurity`

Resources and Processes

Remember in a previous activity, we discovered a suspicious script in `/tmp`.

Since scripts are executable programs, our next step is to check which programs are currently running on the machine to determine if anything malicious is taking place.

When a program runs, it must process data and potentially make changes to the file system. This is why we call a running program a **process**.

- For example, a text-editing application needs to save temporary files to back up users' work, and then save their final draft to a file on disk.

When these programs process, save, and modify data, they consume a computer's **resources**.

- The two important resources we'll look at are **memory** and **the Central Processing Unit (CPU)**.
- As we will see shortly, programs compete to use a finite amount of resources available on a machine.
- This limited amount of resources is something attackers will take advantage of when attacking a machine.

Memory

- Memory comes in two forms: **Random Access Memory (RAM)** and disk space.
- Disk space is used to save data "permanently." In other words, files saved to a disk persist even after a process ends.
- By contrast, RAM is used to run the program's code. In other words, it is only used while the program is running.
- The more work a process does, the more RAM it needs to use. Processes that do a lot of work often use a lot of RAM. However, there is only a finite amount of RAM available, so such massive processes often slow down the computer by snapping up available memory from other ones.

The **CPU** also works to process programs on a system:

- The CPU acts as the central brain of a system, determining how much work a process has to do, and how "difficult" that work is.
- A difficult task (such as encrypting a large file) will use a lot of RAM and/or CPU, while an easier task (like executing the `ls` command) will require minimal amounts of these resources.

Hackers can take advantage of a system's finite resources:

- Linux systems can run multiple processes at once. Since a system only has finite memory and disk space, all of these processes must compete for resources.
- Hackers can perform denial of service (DoS) attacks by launching processes that use up memory on a target machine. This can slow down or crash the machine, making it unavailable to users, and thus denying them service.
- Hackers can also start malicious processes that don't use a lot of memory and are, therefore, not easy to spot without specifically looking for them.
 - One example is a "backdoor" process, which allows cybercriminals to break into machines they shouldn't have access to without being detected. These don't use much memory because they essentially make a network connection to the cybercriminal's machine and then listen for instructions.

- These threats emphasize the importance of being able to identify normal system processes from runaway or malicious processes.

Linux has several commands for managing processes:

- `top` allows you to observe all the running processes in real time. It updates every three seconds to let you know what's happening on the system.
 - `top` is often used by administrators to get a general idea of how the system is running in real time. It is immediately obvious if a process is using too much memory or CPU.
- `ps` allows you to take a snapshot of all the running processes on the system. You can use different arguments to observe different subsets of processes and use this output with other commands in the command line.
 - Snapshots are good if you want to get some process information programmatically, or if you don't need to see how the program is behaving and only want some basic info.
- `kill` is used to stop processes, usually ones that are causing problems. `kill` attempts to allow a process to finish what it's doing before it shuts down. However, we will learn how to use a flag that will immediately stop any process.

Overall, we use `top` and `ps` to get an idea of the overall "system health" of the machine. These commands let us identify details such as how much memory/CPU a process is using and even what user started each process.

- A healthy machine is one in which a system is not being overtaxed. In other words, the machine is not using too much memory, RAM, disk space, or other resources.
- When a system is overtaxed, it affects uptime, which in turn compromises the availability of the services that system provides. A machine only has a finite amount of resources, so when they are used up, the machine slows down and cannot process requests as easily.
- An extreme example of this would be an attacker intentionally using up a system's resources to slow the machine down and compromise its availability. This type of attack is often carried out on a web server, and stops the machine from being able to display its website. It is known as a DoS attack.

We want to run the potentially malicious script to determine how it affects system health. This is usually done by:

- Moving the malicious script or program to a contained environment (such as a virtual machine) where it cannot escape or affect other machines.
- Executing the malicious script or program.
- Monitoring how it uses memory/CPU, which files it accesses, any network communications it makes, along with any other behavior it exhibits.
- This is called **dynamic analysis** and helps us understand an attacker's intent.
- Dynamic analysis is useful for determining how we should harden the system and where else to look for consequences or evidence of compromise.

In real life, you wouldn't run the script right on the machine you're auditing, but for the purposes of this exercise, we will run it on the same machine.

- Note that this machine is already a VM.

In the upcoming demo, we will run the suspicious script we found in the `/tmp` directory during our previous audit. We will then use `top` and `ps` to understand what it does, and use `kill` to end the process after you understand its behavior.

Inspecting Malicious Files Demo

Follow along with these steps from the `student` account.

Log into the Instructor VM environment with the following credentials before proceeding.

- Username: `student`
- Password: `cybersecurity`

First, we'll run `top` on our system to better understand its output.

- Run `top`.

`top` splits its output into two panes: an upper "summary" pane and a lower "details" pane.

```
top - 17:45:13 up 1 day, 4:51, 0 users, load average: 0.16, 0.21, 0.21
Tasks: 207 total, 1 running, 162 sleeping, 1 stopped, 1 zombie
%Cpu(s): 4.3 us, 1.2 sy, 0.0 ni, 93.2 id, 0.5 wa, 0.4 hi, 0.1 si, 0.3 st
KiB Mem : 7956892 total, 546744 free, 4446360 used, 2963788 buff/cache
KiB Swap: 1048572 total, 1048060 free, 512 used. 2947992 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
31554	user	20	0	2102260	504808	206648	S	6.6	6.3	1:20.60	slack
1010	user	20	0	1339916	295392	143484	S	3.7	3.7	19:25.84	brave
706	user	20	0	473756	129772	31412	S	3.3	1.6	19:29.02	Xorg
1047	user	20	0	679156	155824	103140	S	3.0	2.0	16:07.86	brave
1608	user	20	0	2059632	202464	114372	S	2.3	2.5	7:21.20	slack
3721	user	20	0	773488	129468	93236	S	2.0	1.6	0:01.29	brave
2523	user	20	0	5216108	550280	102988	S	1.7	6.9	164:54.60	zoom
3918	user	20	0	1228896	378548	77984	S	1.3	4.8	25:37.72	brave
541	root	20	0	72816	4108	3496	S	1.0	0.1	5:36.15	qubes-gui
1393	user	20	0	1732116	274076	125828	S	0.7	3.4	16:38.75	atom
893	user	20	0	696512	663172	5684	S	0.3	8.3	0:08.52	icon-sender
1646	user	20	0	567020	77032	55912	S	0.3	1.0	0:03.31	slack
2766	user	20	0	1095280	379392	82648	S	0.3	4.8	20:31.15	brave
3995	user	20	0	827964	167592	84400	S	0.3	2.1	2:29.23	brave
27891	user	20	0	44976	3860	3116	R	0.3	0.0	0:00.15	top

Note the following:

- The information in the summary pane summarizes **overall resource usage**.

```
top - 17:45:13 up 1 day, 4:51, 0 users, load average: 0.16, 0.21, 0.21
Tasks: 207 total, 1 running, 162 sleeping, 1 stopped, 1 zombie
%Cpu(s): 4.3 us, 1.2 sy, 0.0 ni, 93.2 id, 0.5 wa, 0.4 hi, 0.1 si, 0.3 st
KiB Mem : 7956892 total, 546744 free, 4446360 used, 2963788 buff/cache
KiB Swap: 1048572 total, 1048060 free, 512 used. 2947992 avail Mem
```

- The first row shows the current time of the system, how long it has been up, and how many users are using the system.
- The second row contains information on how many processes, also known as **tasks**, are currently running.
- The remaining lines in the summary pane show overall CPU and memory usage. **CPU usage** refers to how much the computer has to "work" to finish its task. **Memory usage** refers to how much RAM and disk space are used to do that work.
- A quick glance at the summary pane reveals the general state of the system. High CPU and memory usage indicates that the computer is running inefficiently. The number of active users reveals whether more people than usual are using the machine.

While this summary pane is useful for diagnosing a machine's general health, we must read the details pane to understand which specific processes are using CPU and/or memory.

The details pane breaks down usage by each individual process running on the system.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
31554	user	20	0	2102260	504808	206648	S	6.6	6.3	1:20.60	slack
1010	user	20	0	1339916	295392	143484	S	3.7	3.7	19:25.84	brave
706	user	20	0	473756	129772	31412	S	3.3	1.6	19:29.02	Xorg
1047	user	20	0	679156	155824	103140	S	3.0	2.0	16:07.86	brave
1608	user	20	0	2059632	202464	114372	S	2.3	2.5	7:21.20	slack
3721	user	20	0	773488	129468	93236	S	2.0	1.6	0:01.29	brave
2523	user	20	0	5216108	550280	102988	S	1.7	6.9	164:54.60	zoom
3918	user	20	0	1228896	378548	77984	S	1.3	4.8	25:37.72	brave
541	root	20	0	72816	4108	3496	S	1.0	0.1	5:36.15	qubes-gui
1393	user	20	0	1732116	274076	125828	S	0.7	3.4	16:38.75	atom
893	user	20	0	696512	663172	5684	S	0.3	8.3	0:08.52	icon-sender
1646	user	20	0	567020	77032	55912	S	0.3	1.0	0:03.31	slack
2766	user	20	0	1095280	379392	82648	S	0.3	4.8	20:31.15	brave
3995	user	20	0	827964	167592	84400	S	0.3	2.1	2:29.23	brave
27891	user	20	0	44976	3860	3116	R	0.3	0.0	0:00.15	top

The following columns are the most important sections for our purposes:

- PID:** A number identifying each running process. This is essentially a name for each process. The PID is **extremely important**, as this is how you specify which process to stop when using `kill`.
- USER:** Which user started the process.
- PR:** "Process priority" means the higher the number, the more important the process. This determines how many resources are dedicated to each process and if one action gets processed before another. It is determined by the system.

- **NI:** "Nice value" is related to the process priority. A nice value of -20 means "highest process priority." A nice value of 19 means "lowest process priority." Most processes have a nice value of zero. A process priority can be escalated in a number of ways, though it is usually managed automatically.
- **COMMAND:** This is the name of the command that started the process.
- **%CPU:** This is the percentage of CPU the process is using.
- **%MEM:** This is the percentage of RAM the process is using.

You can enable column highlighting and sorting by pressing the `x` key. By default, the `%CPU` column is highlighted and sorted by highest CPU usage.

```
top - 18:18:03 up 1 day, 5:24, 0 users, load average: 0.37, 0.37, 0.28
Tasks: 205 total, 4 running, 158 sleeping, 1 stopped, 1 zombie
%Cpu(s): 1.5 us, 0.3 sy, 0.0 ni, 96.5 id, 1.1 wa, 0.3 hi, 0.0 si, 0.2 st
KiB Mem : 7956892 total, 429992 free, 4573212 used, 2953688 buff/cache
KiB Swap: 1048572 total, 1048060 free, 512 used. 2827552 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1608	user	20	0	2059424	202536	114424	S	1.3	2.5	7:28.70	slack
2523	user	20	0	5216108	550288	102988	S	1.3	6.9	165:23.85	zoom
3918	user	20	0	1228276	384356	77984	S	1.3	4.8	26:03.45	brave
31554	user	20	0	2106660	509480	206816	S	1.3	6.4	1:27.28	slack
706	user	20	0	475228	131264	31432	R	0.9	1.6	19:44.91	Xorg
31040	user	20	0	2140804	524168	223592	S	0.9	6.6	1:37.23	slack
824	user	20	0	362036	14452	5456	S	0.4	0.2	0:42.15	ibus-daemon
844	user	20	0	457720	38084	31928	S	0.4	0.5	0:00.19	ibus-x11
1010	user	20	0	1330584	294588	142408	S	0.4	3.7	19:39.44	brave
1310	user	20	0	784164	38024	27776	S	0.4	0.5	0:05.01	gnome-terminal-
1393	user	20	0	1732244	272492	125616	S	0.4	3.4	17:14.06	atom
2766	user	20	0	1113588	398856	91244	S	0.4	5.0	21:12.34	brave
30324	user	20	0	2120160	513804	225044	S	0.4	6.5	0:22.35	slack
1	root	20	0	204740	6276	4552	S	0.0	0.1	0:01.16	systemd

You can also sort the processes by what user started it by pressing the `u` key:

- Press the `u` key.
- Enter `root` to demonstrate sorting by processes started by the root user.

```
top - 15:54:59 up 2 days, 3:01, 0 users, load average: 0.18, 0.16, 0.17
Tasks: 204 total, 1 running, 159 sleeping, 2 stopped, 1 zombie
%Cpu(s): 0.4 us, 0.1 sy, 0.0 ni, 98.2 id, 1.0 wa, 0.2 hi, 0.0 si, 0.1 st
KiB Mem : 7956892 total, 1671872 free, 4333180 used, 1951840 buff/cache
KiB Swap: 1048572 total, 1047804 free, 768 used. 3356092 avail Mem
Which user (blank for all) root
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	204740	6336	4580	S	0.0	0.1	0:01.42	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.01	kthreadd
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	mm_percpu_wq
7	root	20	0	0	0	0	S	0.0	0.0	0:00.23	ksoftirqd/0
8	root	20	0	0	0	0	I	0.0	0.0	0:18.54	rcu_sched
9	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_bh
10	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	migration/0
11	root	rt	0	0	0	0	S	0.0	0.0	0:00.13	watchdog/0
12	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
13	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
14	root	rt	0	0	0	0	S	0.0	0.0	0:00.20	watchdog/1
15	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	migration/1
16	root	20	0	0	0	0	S	0.0	0.0	0:00.06	ksoftirqd/1
18	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/1:0H
19	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/2
20	root	rt	0	0	0	0	S	0.0	0.0	0:00.22	watchdog/2
21	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	migration/2
22	root	20	0	0	0	0	S	0.0	0.0	0:00.09	ksoftirqd/2
24	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/2:0H

At the moment, nothing much is happening on the system, so CPU usage is relatively low. This is how you'd expect a system to usually appear.

You can close `top` by pressing `q`.

- Press `q` to return to the command line.

`Top` generates a live display of all running processes, which includes a lot of information.

Sometimes all you need is a "snapshot" of which processes are running and their PIDs, so you can stop those that are causing problems.

Note that `ps` is great if you want to look at only one process or a few processes and get their PID to interact with them or shut them down. Because it outputs a static list of processes, it is also useful for interacting with processes programmatically.

While not as powerful as `top`, `ps` is a "quick and dirty" way to get information about running processes.

- Run `ps`.

Your output should be similar to the following:

Unset

```
$ ps
```

PID	TTY	TIME	CMD
1319	pts/0	00:00:00	bash
29243	pts/0	00:00:00	ps

- This lists the PID, TIME, and CMD (command).
- The `bash` process is the shell you're using.
- The `ps` process is the command you just ran.

Note that `ps` can generate more detailed output with the addition of special flags.

- Type `ps a`.

Your output should be similar to the following:

Unset

PID	TTY	STAT	TIME	COMMAND
625	hvc0	Ss+	0:00	/sbin/agetty --keep-baud
115200,38400,9600	hvc0 vt220			
627	tty1	Ss+	0:00	/sbin/agetty --noclear tty1 linux
1319	pts/0	Ss	0:00	bash
3692	pts/0	T	0:00	top
17959	pts/0	R+	0:00	ps a
27778	pts/1	Ss	0:00	bash

The syntax can be broken down as follows:

- `ps` is the process command.
- `a` stands for *all* processes that have a terminal assigned to them.

A "terminal" refers to the command line terminal that we are using.

- In the past, terminals used to be one physical screen or monitor called a "teletype terminal" or TTY terminal. Today, terminals are still referred to as a TTY terminal, but they are virtual. A Linux system has multiple virtual TTY terminals.

Some processes are assigned a virtual TTY terminal in case you need to interact with that process later.

- Type `ps aux`.

The syntax can be broken down as follows:

- `ps a` is the same as the previous command.
- `u` selects by effective user ID.
- `x` lists processes that do not have a terminal assigned to them.

Note that we do not need to start these options with `-` because `ps` is a very old program written before the `-` standard for options was adopted.

When used together, these three options take a snapshot of every process running within a few milliseconds of you pressing the Enter key.

- Run `ps aux`.

Output should look like the following:

```
Unset
$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME
COMMAND									
root	1	0.0	0.0	204740	6276	?	Ss	Jul24	0:01
/sbin/init									
root	2	0.0	0.0	0	0	?	S	Jul24	0:00
[kthreadd]									
root	4	0.0	0.0	0	0	?	I<	Jul24	0:00
[kworker/0:0H]									
root	6	0.0	0.0	0	0	?	I<	Jul24	0:00
[mm_percpu_wq]									
root	7	0.0	0.0	0	0	?	S	Jul24	0:00
[ksoftirqd/0]									
root	8	0.0	0.0	0	0	?	I	Jul24	0:12
[rcu_sched]									
root	9	0.0	0.0	0	0	?	I	Jul24	0:00
[rcu_bh]									
root	10	0.0	0.0	0	0	?	S	Jul24	0:00
[migration/0]									
root	11	0.0	0.0	0	0	?	S	Jul24	0:00
[watchdog/0]									
root	12	0.0	0.0	0	0	?	S	Jul24	0:00
[cpuhp/0]									
root	13	0.0	0.0	0	0	?	S	Jul24	0:00
[cpuhp/1]									

```

root          14  0.0  0.0      0      0 ?        S    Jul24   0:00
[watchdog/1]
root          15  0.0  0.0      0      0 ?        S    Jul24   0:00
[migration/1]
root          16  0.0  0.0      0      0 ?        S    Jul24   0:00
[ksoftirqd/1]
root          18  0.0  0.0      0      0 ?        I<   Jul24   0:00
[kworker/1:0H]
root          19  0.0  0.0      0      0 ?        S    Jul24   0:00
[cpuhp/2]
...

```

- This output lists all processes sorted by the user that started them. It gives you a snapshot of *all* of the processes running on the machine at the time you ran the command.
- This command is great if you want to review every process running from all users and potentially investigate everything from a high level.
- You can use other command line programs like `cut`, `grep`, or `less` to make this output more manageable.

Filter the output for any line that contains `bash`.

- Run `ps aux | grep bash`.

Your output should be similar to the following:

```

Unset
user          1319  0.0  0.0  21604  5520 pts/0    Ss   Jul24   0:00
bash
user          27778  0.0  0.0  21488  5360 pts/1    Ss   17:44   0:00
bash
user          29979  0.0  0.0  12720   936 pts/0    S+   18:56   0:00
grep bash

```

This output is much more manageable than sorting through every process manually.

Instructor Demo

Now that we understand the information report `top`, we'll run our potentially malicious script to see if that information changes. This is an indication that the script is causing the system to run abnormally.

Note: Watch this demo rather than following along.

- Open two terminals.
- Locate the suspicious `a9xk.sh` script in your `~/research` directory.
- Run `cat /home/instructor/Documents/research/a9xk.sh`.
 - This script is using the `stress` command, so you will be looking for the `stress` process to kill it.
- Run `sudo bash ~/Documents/research/a9xk.sh` in one of the terminals. The script runs a `sudo` command, so you may need to enter a password.
- In the second terminal, open `top` again.
- Press `x` again to sort by `%CPU`. Your resulting output should be similar to the following image:

```
top - 14:21:49 up 1 day, 1:20, 2 users, load average: 1.08, 0.91, 1.01
Tasks: 221 total, 10 running, 167 sleeping, 0 stopped, 0 zombie
%Cpu(s): 83.4 us, 3.2 sy, 0.0 ni, 0.0 id, 0.0 wa, 1.5 hi, 0.2 si, 11.7 st
KiB Mem : 7956892 total, 1881092 free, 3470856 used, 2604944 buff/cache
KiB Swap: 1048572 total, 1048572 free, 0 used. 3927692 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
16247	root	20	0	7276	88	0	R	45.5	0.0	0:02.58	stress
16251	root	20	0	7276	88	0	R	44.5	0.0	0:02.77	stress
16245	root	20	0	7276	88	0	R	43.9	0.0	0:02.32	stress
16242	root	20	0	7276	88	0	R	43.2	0.0	0:02.35	stress
16243	root	20	0	7276	88	0	R	43.2	0.0	0:02.40	stress
16240	root	20	0	7276	88	0	R	41.9	0.0	0:02.25	stress
16248	root	20	0	7276	88	0	R	40.9	0.0	0:02.28	stress
16249	root	20	0	7276	88	0	R	39.2	0.0	0:02.50	stress
16250	root	20	0	7276	88	0	R	37.5	0.0	0:02.39	stress
16241	root	20	0	7276	88	0	D	2.7	0.0	0:00.13	stress
16244	root	20	0	7276	88	0	D	2.3	0.0	0:00.11	stress
16246	root	20	0	7276	88	0	D	2.3	0.0	0:00.11	stress

- You can easily see the `stress` command in the command column.
- The `%CPU` column reveals that this process is using a substantial amount of resources.

Use the `<` and `>` keys to sort by the CPU column. Then, sort by the Memory column.

- CPU usage is very high, but memory is relatively low.
- This is because the malicious script causes the computer to perform a lot of computations, but does not require much memory to do them.
- This demonstrates that *either* high CPU usage *or* high memory consumption can slow down a computer.
- Security professionals and systems administrators should inspect both values to accurately identify why a machine might be running slowly.

Upon identifying a suspicious process, the natural next steps include:

1. Identifying the user who started it; and
2. Killing the process.

In this case, we can see that the `stress` command was started by `root`, because the script uses the `sudo` command. However, we'll demonstrate how to search for processes started by a particular user:

- Press `u` to get a prompt in between the two panes.
- Enter `sysadmin` as the username to filter for your own processes.
- Note that `stress` does not appear, since it was started by `root`.

Note that we cannot observe the process for `a9xk.sh` here.

Remember that `top` contains the PID for the `stress` commands and this is all the information you need to stop them. However, for the sake of demonstration, we'll use `ps aux` to give us this information.

- Press `q` to close `top`.
- Run `ps aux | less` to get a snapshot of all running processes in a more navigable output.

Looking at *all* the processes this way is a bit cumbersome.

- Run `ps a` and note that this gives us a much more manageable output and we can observe the `stress` processes clearly.

Note one of the `stress` PIDs you want to stop. Remember that the PID is the second column from the left.

Once we know what process we are checking for, we can now use the `kill` command to stop it. Running `kill <PID>` will send a signal to that process to stop what it is doing and return resources to the system before exiting.

- Run `sudo kill <PID for stress>`.
- Run `ps a` again to prove that the process has been stopped.

This might take a long time if you have to stop each one individually.

Note that some programs launch several processes:

- Some programs must launch several processes in order to do their jobs. For example, an application like Spotify might need to start a network process to stream music, as well as a GUI process to display the user interface.
- If a program has several processes going, administrators can use `killall <process name>` to tell the OS to stop the target process and any sub-processes it has started.
- This command is also good if you do not know the PID of the specific process to stop, but do know the name of the application you want to close.

Run `sudo killall stress`.

- Run `ps a` again to demonstrate that all the stress processes are stopped.
- Finally, run `top`. Its output now looks like it did originally.

Quitting those processes brought CPU usage back down, thus allowing the machine to run faster.

While this example is simple for ease of demonstration, the `kill` processes is important for controlling exactly what is running on a host, which is important for both sysadmins and security professionals.

Activity - Process Investigation

- [Activity File: Process Investigation](#)
-

Installing Packages

While a default Linux build comes with many useful programs, there are often times when the tool or program you need is not installed on your machine. When this happens, you will have to install the program yourself.

You are likely used to using GUI interfaces for this installation and downloading files from either a website or app store.

In Linux, this can all be done from the terminal through the use of a `package manager`, which has the following important features:

- They simplify the installation and management of software.
- Ensure that software dependencies are handled automatically.
- Provide a centralized system for software updates and security patches.

Repositories (repos) are collections of software packages that package managers pull from. These repositories are maintained by the Linux distribution or third-party organizations, ensuring that the software is secure and compatible with the system.

Note: Official repositories help to protect the system by only offering authorized software. This does not guarantee that the software is free of malware, only that it has been vetted in an official repository.

Package Manager

A package manager is a tool that automates the process of installing, upgrading, configuring, and removing software packages.

The role of a package manager is to simplify software management. Popular package managers include `apt` (Debian-based), `yum` (Red Hat-based), and `pacman` (Arch Linux).

Because we are using Ubuntu, which is a Debian-based build, we will be using `apt` as our package manager.

Repositories

Repositories (or repos) are collections of software packages that are available for installation.

The different types of repositories include:

- **Official repositories:** Maintained by the Linux distribution (e.g., Ubuntu's official repositories).
- **Third-party repositories:** Maintained by other organizations or individuals.
- **Personal package archives (PPAs):** User-maintained repositories that can be added to the system.

Introduction to APT

In our weblabs, we will be using `apt` as our package manager.

Introduction to APT

APT (advanced package tool) is the package manager used by Debian-based distributions like Ubuntu.

Using the terminal, walk through the following commands.

Basic APT Commands

The following activities are run with `sudo` and installations require `root` access to run.

`sudo apt update`: Updates the list of available packages and their versions.

Unset

```
sudo apt update
```

This command does not upgrade any packages, it only updates the package list. This step is important to ensure that any programs being installed are the latest packages.

`sudo apt upgrade`: Installs the newest versions of all packages currently installed.

Unset

```
sudo apt upgrade
```

This command will update all programs currently installed.

`sudo apt install <package_name>`: Installs a specified package.

Unset

```
sudo apt install vim
```

It is important to verify the name of the program before installing it.

`sudo apt remove <package_name>`: Removes a specified package.

Unset

```
sudo apt remove vim
```

This functions like running an uninstaller from the GUI.

`sudo apt search <package_name>`: Searches for a package in the repositories.

Unset

```
sudo apt search vim
```

This command can be used if you are not certain of the actual package name.

Automating Installs

Note that whenever upgrading Linux packages, the system will identify the packages that are being updated and how much disk space will be used before waiting for confirmation. While learning Linux, this is a good practice, but in a production environment, sometimes you will be updating 100's of machines and waiting for confirmation will take much longer.

Package managers have thought of this and added the `-y` option to the commands.

`sudo apt install vim -y` will automatically assume that you are clicking yes on installs and run the install without confirmation.

Installing Packages

Follow along on your own machines.

While we have been doing everything in the default terminal window, this is not the only option for command line access in Linux.

While simple and built-in, the default terminal has a number of limitations that make it less practical for IT professionals.

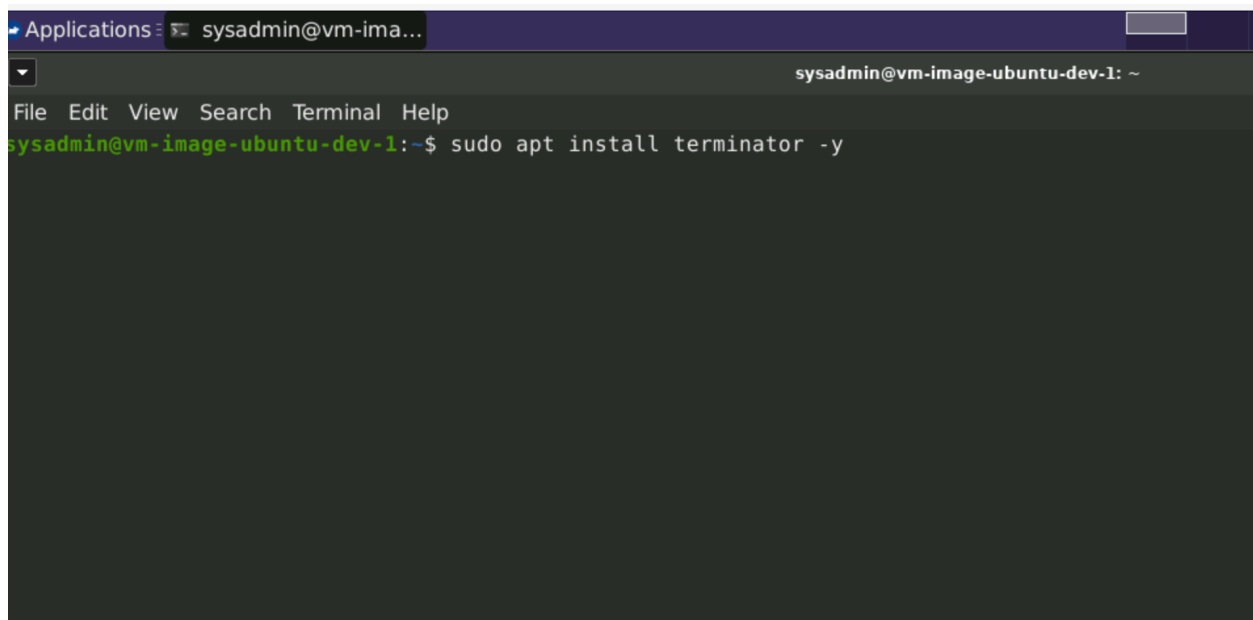
- **Single window:** Terminal is limited to one window at a time and managing multiple processes requires multiple terminal instances.
- **Basic features:** Terminal lacks extensive customization options or multipane support.

`Terminator` is an advanced terminal emulator that supports multiple terminals in a single window. It is designed for users who need to manage multiple terminal sessions simultaneously.

In addition to acting as a command line, `Terminator` offers the following features:

- Multiple panes: Allows splitting the terminal window into multiple panes, enabling users to work on several tasks simultaneously within a single window.
- Tabs: Supports tabbed terminals for organizing different sessions.
- Customization: Extensive customization options, including themes, fonts, and keybindings.
- Broadcasting: Can broadcast input to multiple terminals, useful for running the same command across several sessions.
- Search: Provides an in-terminal search function to find specific text within the terminal.

`Terminator` can be installed by running the command `sudo apt install terminator -y`.

A screenshot of a terminal window. The title bar at the top says "Applications: sysadmin@vm-ima...". The terminal itself has a dark background with a menu bar (File, Edit, View, Search, Terminal, Help) and a status bar at the bottom that says "sysadmin@vm-image-ubuntu-dev-1: ~". The command prompt shows "sysadmin@vm-image-ubuntu-dev-1:~\$" followed by the command "sudo apt install terminator -y" which has been entered.

- This will install Terminator with all its dependencies.

- Launch Terminator from the terminal with the command `terminator`.
 - This will open a terminal inside of Terminator.
- Right-click anywhere in the terminal and select `split horizontally` or `split vertically` and demonstrate the additional panes within the main window.



Each pane is running a separate process so IT professionals can run different commands under different user accounts or run `top` or `ps` in one window while demonstrating how different commands affect those processes.

Installing Packages

- [Activity File: Installing Packages](#)